Informe del proyecto · VetCare DB

Proyecto Integrador · Bases de Datos II

1. Portada

Grupo 5

integrantes

Monica Daysure Ortiz Ossa

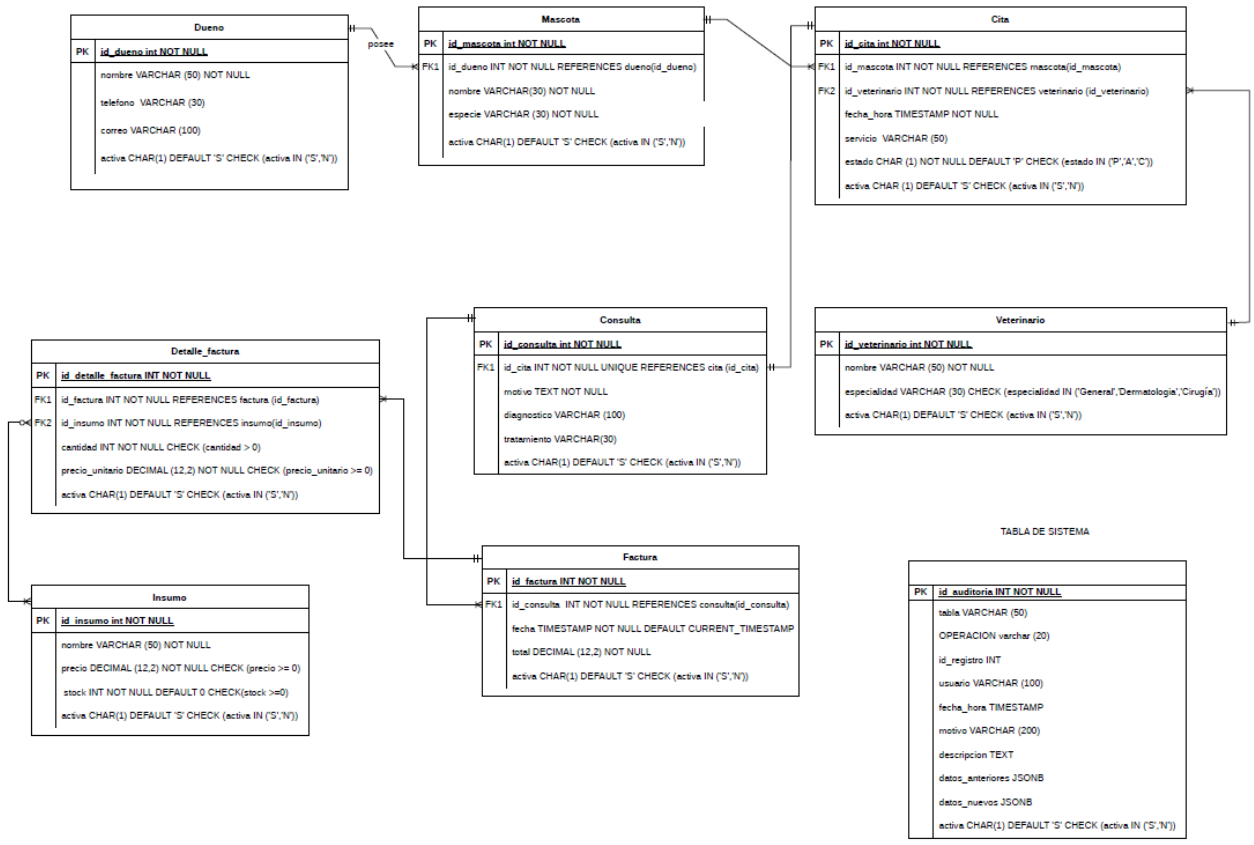
Anderson Guapacho Gaitán

Curso Base de Datos

Grupo 641ª

14/11/2026

2. Modelo de datos



Justificación del modelo

Decisiones de diseño del modelo Entidad-Relación:

1. **Entidades y Relaciones:** El modelo refleja fielmente el flujo operativo de la clínica mediante ocho tablas principales (desde el dueño hasta la factura), asegurando que una cita conecte a la mascota con el veterinario, y que la consulta médica dependa estrictamente de una cita (relación 1 a 1 usando un UNIQUE).

2. **Llaves Foráneas:** Cada llave foránea se ubicó en el lado "muchos" de la relación para garantizar la integridad referencial; además, se separó el detalle de la factura para guardar el precio_unitario histórico, evitando que una factura antigua cambie si el precio del insumo se actualiza hoy.
3. **Restricciones (Reglas de negocio):** Se implementaron restricciones CHECK para dominios cerrados (como las especialidades o los estados de la cita) y para la Regla 2 (stock >= 0), y se usó la baja lógica con el campo activa en lugar de borrados físicos para no perder el historial clínico.
4. **Tabla de Auditoría:** Se agregó una novena tabla de sistema llamada auditoria (aislada de las relaciones del negocio), la cual es llenada automáticamente por los disparadores para cumplir con la Regla 3 de dejar rastro de quién y cuándo se modifican precios o se cancelan citas.
5. **Convenciones del curso:** Todo el modelo respeta el estándar solicitado: nombres en singular y minúscula, tipos de datos precisos (dinero en DECIMAL(12,2), fechas en TIMESTAMP y teléfonos en VARCHAR(30)), y llaves primarias sustitutas con el patrón id_entidad.

3. Seguridad y respaldo

Lo que puede y no puede hacer cada rol. Debajo de la tabla, el plan de respaldo y recuperación: cada cuánto se respalda, cómo se restaura y cuántos datos se pueden perder como máximo.

Rol	Qué puede hacer	Qué no puede hacer
rol_admin (Dueño/administrador)	Leer todas las tablas (incluyendo métricas y auditoría). Insertar y actualizar precios y stock de insumos. Ejecutar todos los procedimientos y funciones.	Borrar registros (no tiene DELETE ni TRUNCATE). Modificar o borrar la tabla de auditoría. Insertar citas directamente (debe usar el procedimiento sp_agendar_cita).
rol_operativo	Leer dueños, mascotas, citas, consultas, insumos y facturas. Registrar dueños, mascotas, consultas y facturas. Agendar y cancelar citas usando los procedimientos.	Cambiar precios o stock de insumos. Leer la tabla de auditoría. Insertar citas o modificar su estado directamente. Borrar cualquier registro.

Plan de respaldo y recuperación

- **Frecuencia:** Se realiza un respaldo lógico completo todos los días a las 20:30 (después del cierre de la clínica) utilizando pg_dump -Fc, y una copia base semanal con pg_basebackup.
- **Almacenamiento:** Se aplica la regla 3-2-1: dos copias en el servidor local de la clínica y una copia externa en la nube (AWS S3 o Google Drive), garantizando que un daño físico en el servidor no destruya la información.
- **Restauración:** En caso de fallo, se crea una base de datos limpia y se ejecuta pg_restore con el último archivo. Si se requiere recuperar hasta un minuto exacto (por ejemplo, antes de un borrado accidental), se restauran los registros WAL (Write-Ahead Logging).
- **RPO (Pérdida máxima de datos):** Aceptamos perder como máximo 5 minutos de operación (usando archivado continuo de WAL) o 24 horas si dependemos solo del dump nocturno.
- **RTO (Tiempo máximo de recuperación):** La base de datos debe volver a estar en línea en un máximo de 1 hora.

- **Verificación:** Una vez al mes se restaura el respaldo en un servidor de pruebas (vetcare_restore) para confirmar que los archivos no están corruptos.

4. Objetos programables

Cada procedimiento, función y disparador, con su caso de prueba y lo que devolvió.

Objeto	Tipo	Qué hace	Caso de prueba	Resultado
sp_agendar_cita	Procedimiento	Valida que la mascota y el veterinario estén activos, que la fecha sea futura y que no haya choque de agenda. Luego inserta la cita.	Agendar a Toby (mascota 4) con Ivan Ortiz (vet 3). Luego intentar agendar a Rocky (mascota 3, inactiva).	Toby se agenda exitosamente. Rocky es rechazada con el error: "La mascota 3 esta inactiva: no se le pueden agendar citas nuevas".
sp_cancelar_cita	Procedimiento	Cancela una cita pendiente cambiando su estado a 'C' en lugar de borrarla, y guarda el motivo de la cancelación.	Cancelar una cita recién agendada y luego intentar cancelarla de nuevo.	La primera vez cambia a estado 'C'. La segunda vez arroja error: "La cita ya esta cancelada".
fn_total_factura	Función	Recorre la tabla detalle_factura y suma (cantidad × precio_unitario) de las líneas activas para calcular el total a cobrar.	Ejecutar SELECT fn_total_factura(1); sobre la factura de prueba.	Retorna **121000.00** (correspondiente a la suma de los insumos de esa factura).
trg_auditoria	Disparador (Regla 3)	Se activa cuando se cambia el precio de un insumo o se modifica el estado de una cita. Guarda en la tabla auditoria quién lo hizo, cuándo y los valores antes/después.	Actualizar el precio del insumo 1 a 50000. Luego consultar la tabla auditoria.	Se inserta una fila con el usuario, la fecha y un JSONB que muestra el precio anterior y el nuevo.
trg_detalle_factura_stock	Disparador (Regla 2)	Se activa al insertar una línea en detalle_factura. Descuenta el stock del insumo y, si no hay suficiente, lanza un error abortando la factura.	Insertar una línea pidiendo 9999 unidades del insumo 5 (que solo tiene 7 en stock).	El sistema aborta la operación con el error: "Stock insuficiente de Shampoo medicado: disponible 7, solicitado 9999".

5. Optimización

Consulta	Costo antes	Qué se cambió	Costo después
1. Conteo de citas en julio de 2026	Seq Scan sobre 30.010 citas (lee toda la tabla). Tiempo: **21.468 ms**.	Se reescribió el filtro. Se pasó de usar EXTRACT(YEAR/MONTH) a un rango de fechas (>= '2026-07-01' y < '2026-08-01'). Además, se creó el índice ix_cita_fecha_hora.	Index Only Scan. Tiempo: **4.639 ms** (4.6 veces más rápido).
2. Mascotas por cada dueño (Bono)	Subconsulta correlacionada (se ejecutaba una vez por cada uno de los 2000 dueños). Tiempo: **42.303 ms**.	Se reescribió la consulta reemplazando la subconsulta por un LEFT JOIN con la tabla de mascotas y un GROUP BY.	Hash Left Join y Aggregate (una sola pasada por las tablas). Tiempo: **9.018 ms** (4.7 veces más rápido).

¿Por qué mejora cada consulta?

- **En la Consulta 1:** El problema original era que usar funciones como EXTRACT() sobre una columna de fecha (envolver la columna) obliga a PostgreSQL a evaluar la función fila por fila, por lo que **el motor no puede usar ningún índice** y se ve forzado a hacer un escaneo secuencial (Seq Scan). Al reescribir la consulta como un rango (sargable) y crear el índice ix_cita_fecha_hora, el motor lee únicamente las filas de ese mes. Aquí hicieron falta ambas cosas: sin la reescritura el índice se ignora, y sin el índice la reescritura no ayuda.
- **En la Consulta 2:** Una subconsulta correlacionada se paga una vez por cada fila de la tabla externa (en este caso, 2.000 veces). Al convertirla en un LEFT JOIN con GROUP BY, el motor utiliza algoritmos de unión en memoria (Hash Join), recorriendo la tabla dueno y la tabla mascota una sola vez, lo que reduce drásticamente el tiempo de respuesta. Esta es la consulta que el veterinario lanzaría en plena consulta para ver cuántas mascotas tiene un dueño.

6. Integración app ↔ BD

Operación	Recibe	Devuelve	Procedimiento que la cumple
Agendar una nueva cita	ID de la mascota, ID del veterinario, fecha/hora (TIMESTAMP) y el servicio.	El ID de la cita recién creada (o el mensaje de error si falla la regla).	sp_agendar_cita
Cancelar una cita	El ID de la cita y el motivo de la cancelación.	Ninguno (retorna un mensaje de éxito). El registro queda guardado en la tabla de auditoría.	sp_cancelar_cita
Calcular el total a cobrar	El ID de la factura.	El valor total de la factura en formato decimal (DECIMAL(12,2)).	fn_total_factura

Ejemplo de llamada en SQL (desde la aplicación):

```
-- 1. La recepcionista agenda una cita (revisión para la mascota 4 con el veterinario 3)
CALL sp_agendar_cita(4, 3, CURRENT_TIMESTAMP + INTERVAL '2 days', 'Revision', NULL);

-- 2. El dueño cancela la cita (el ID 11 fue el que devolvió la operación anterior)
CALL sp_cancelar_cita(11, 'El dueño reprogramó el viaje');

-- 3. El sistema calcula el total de la factura antes de imprimirla
SELECT fn_total_factura(1);
```

7. Cómo correr los scripts

Requisitos previos:

- Tener instalado **PostgreSQL** (versión 14 o superior).
- Se requiere un usuario con privilegios de superusuario o permisos para crear roles en el servidor (por ejemplo, el usuario postgres), ya que los roles son globales y no pertenecen a una base de datos específica.

Orden de ejecución:

1. Abrir una consola o herramienta como **pgAdmin** o **VS Code**.
2. Crear una base de datos vacía con el comando: CREATE DATABASE bd_VetCare; y conectarse a ella.
3. Ejecutar los archivos **en el siguiente orden estricto** (sin modificar el código):
    - 01_ddl.sql: Crea las tablas, restricciones, llaves foráneas y carga automáticamente los datos de prueba del elenco del anexo.
    - 02_roles_y_privilegios.sql: Crea los dos roles (rol_admin y rol_operativo), asigna los GRANT/REVOKE y ejecuta las pruebas de seguridad.
    - 03_programables.sql: Crea los 2 procedimientos, la 1 función y los 2 disparadores. Al final, ejecuta las pruebas automáticas y hace un ROLLBACK para no ensuciar la base.
    - 04_optimizacion.sql: Inserta un volumen de 30.010 citas (Clase 7) de forma temporal para medir los EXPLAIN antes y después, y al final revierte los datos dejando solo el índice creado.